Student Marks Database

End-to-End MySQL Project

ER Diagram • Schema • Sample Data • Indexes • Views • Stored Procedures • Triggers • Query Bank with Answers

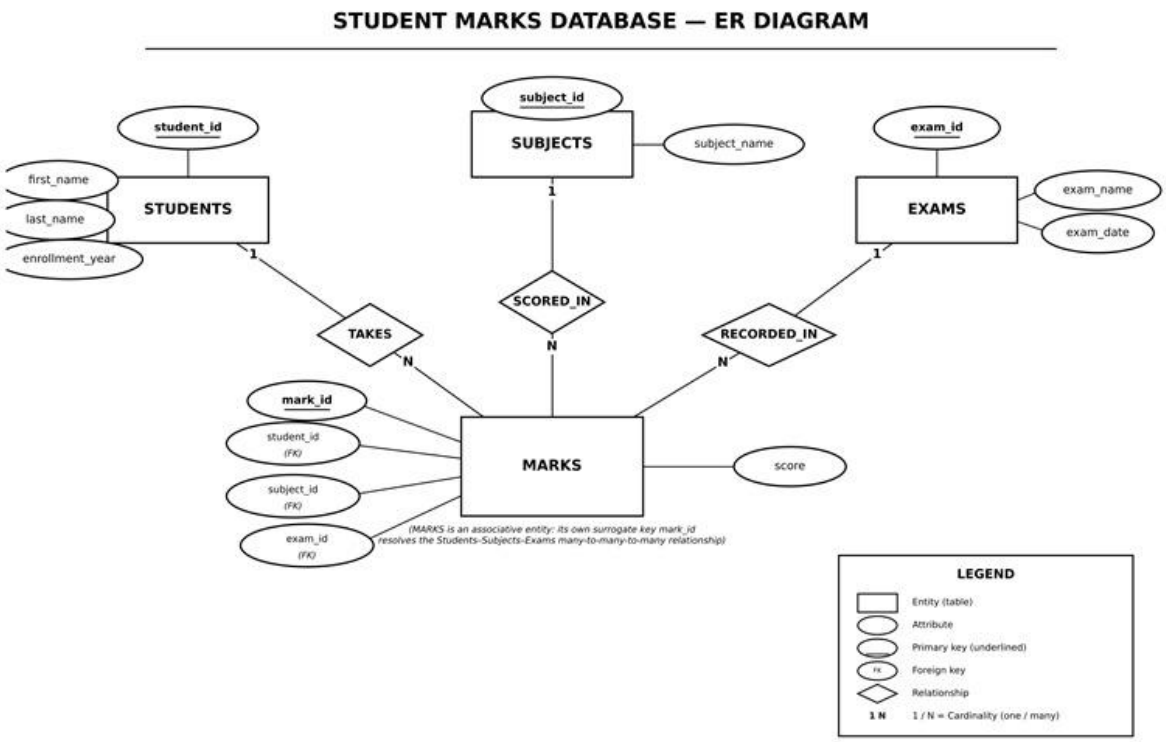
Database: studentdb | RDBMS: MySQL

Table of Contents

- 1. Entity-Relationship (ER) Diagram
- PART 1: DATABASE & TABLE CREATION
- PART 2: SAMPLE DATA
- PART 3: INDEXES
- PART 4: VIEWS
- PART 5: STORED PROCEDURES
- PART 6: TRIGGERS
- PART 7: QUERY BANK - LEVEL 1 : SIMPLE QUERIES (Original Q1-Q3)
- PART 8: QUERY BANK - LEVEL 2 : INTERMEDIATE QUERIES (Original Q4-Q6)
- PART 9: QUERY BANK - LEVEL 3 : COMPLEX QUERIES (Original Q7-Q9)
- PART 10: QUERY BANK - NEW QUERIES (Q10-Q21)
- PART 11: DEMO CALLS - Views / Stored Procedures / Triggers

1. Entity-Relationship (ER) Diagram

STUDENTS, SUBJECTS and EXAMS are independent (strong) entities, each with its own primary key. MARKS is an associative entity: it carries its own surrogate key (mark_id) and three foreign keys, resolving the natural many-to-many-to-many relationship between a student, a subject and an exam.



1.1 Entity Summary

Entity	Type	Primary Key	Notes
STUDENTS	Strong	student_id	Independent entity; a learner enrolled in a given year.
SUBJECTS	Strong	subject_id	Independent entity; a course/subject offered.
EXAMS	Strong	exam_id	Independent entity; a specific exam sitting with a date.
MARKS	Associative (strong, own PK)	mark_id	Foreign keys to all three: student_id, subject_id, exam_id. Represents one recorded score.

2. Project Overview

STUDENT MARKS DATABASE | END-TO-END MYSQL PROJECT

RDBMS : MySQL 8.0+ (also compatible with MariaDB 10.x)

Notes : Every ANSWER below each query is a comment showing what

that query returns when run against the sample data in

Contents:

- PART 1 : Database & Table Creation (schema)
- PART 2 : Sample Data (INSERT statements)
- PART 3 : Indexes
- PART 4 : Views
- PART 5 : Stored Procedures
- PART 6 : Triggers (+ supporting marks_log audit table)
- PART 7 : Query Bank - Level 1 (Original Q1-Q3) + Answers
- PART 8 : Query Bank - Level 2 (Original Q4-Q6) + Answers
- PART 9 : Query Bank - Level 3 (Original Q7-Q9) + Answers
- PART 10 : Query Bank - New Queries (Q10-Q21) + Answers
- PART 11 : Demo calls for Views / Procedures / Triggers
- PART 2. Run the SELECT yourself to confirm.

DATABASE & TABLE CREATION

```
DROP DATABASE IF EXISTS studentdb;
CREATE DATABASE studentdb;
USE studentdb;

-- ---------- Strong Entity: STUDENTS ----------
CREATE TABLE Students (
    student_id      INT PRIMARY KEY,
    first_name      VARCHAR(50),
    last_name       VARCHAR(50),
```

```
    enrollment_year INT
);

-- ---------- Strong Entity: SUBJECTS ----------
CREATE TABLE Subjects (
    subject_id   INT PRIMARY KEY,
    subject_name VARCHAR(50)
);

-- ---------- Strong Entity: EXAMS ----------
CREATE TABLE Exams (
    exam_id   INT PRIMARY KEY,
    exam_name VARCHAR(50),
    exam_date DATE
);

-- ---------- Associative Entity: MARKS ----------
-- Resolves the natural many-to-many-to-many relationship between
-- STUDENTS, SUBJECTS and EXAMS. It has its own surrogate key (mark_id),
-- so it is a strong/associative entity, not a weak entity -- but every
-- row is meaningless without its three foreign keys.
CREATE TABLE Marks (
    mark_id    INT PRIMARY KEY,
    student_id INT,
    subject_id INT,
    exam_id    INT,
    score      INT,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
```

```
    FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id),
    FOREIGN KEY (exam_id)    REFERENCES Exams(exam_id)
);
```

SAMPLE DATA

```
-- Students
INSERT INTO Students VALUES
(1, 'Aarav',  'Sharma', 2023),
(2, 'Diya',   'Patel',  2023),
(3, 'Vihaan', 'Rao',    2024),
(4, 'Ananya', 'Singh',  2024);

-- Subjects
INSERT INTO Subjects VALUES
(101, 'Mathematics'),
(102, 'Science'),
(103, 'Hindi Literature');

-- Exams
INSERT INTO Exams VALUES
(1, 'Half-Yearly 2024',  '2024-09-15'),
(2, 'Final Annual 2024', '2024-03-20');

-- Marks
INSERT INTO Marks VALUES
(1, 1, 101, 1, 85),  -- Aarav,  Math,    Half-Yearly
```

```
(2, 1, 102, 1, 90),  -- Aarav,  Science, Half-Yearly
(3, 2, 101, 1, 75),  -- Diya,   Math,    Half-Yearly
(4, 2, 102, 1, 60),  -- Diya,   Science, Half-Yearly
(5, 3, 101, 1, 95),  -- Vihaan, Math,    Half-Yearly
(6, 1, 101, 2, 88),  -- Aarav,  Math,    Final
(7, 2, 101, 2, 79),  -- Diya,   Math,    Final
(8, 3, 101, 2, 92);  -- Vihaan, Math,    Final

-- Note: Ananya Singh (student_id 4) has no marks recorded, and
-- Hindi Literature (subject_id 103) is never used -- both are useful
-- edge cases for the LEFT JOIN / "never took an exam" queries below.
```

INDEXES

```
-- Foreign-key and frequently-filtered columns get explicit indexes to
-- speed up the JOINs, WHERE and GROUP BY clauses used throughout.

CREATE INDEX idx_marks_student     ON Marks(student_id);
CREATE INDEX idx_marks_subject     ON Marks(subject_id);
CREATE INDEX idx_marks_exam        ON Marks(exam_id);
CREATE INDEX idx_marks_score       ON Marks(score);
CREATE INDEX idx_students_enrollment ON Students(enrollment_year);
CREATE INDEX idx_exams_date        ON Exams(exam_date);

-- Verify indexes:
-- SHOW INDEX FROM Marks;
-- SHOW INDEX FROM Students;
```

VIEWS

```
-- 4.1 Flattened mark + student + subject + exam info (for reporting/UI)
CREATE OR REPLACE VIEW view_student_marks AS
SELECT
        M.mark_id,
        CONCAT(St.first_name, ' ', St.last_name) AS student_name,
        Sb.subject_name,
        E.exam_name,
        E.exam_date,
        M.score
FROM Marks M
JOIN Students St ON M.student_id = St.student_id
JOIN Subjects Sb ON M.subject_id = Sb.subject_id
JOIN Exams E   ON M.exam_id           = E.exam_id;

-- 4.2 Average score per subject
CREATE OR REPLACE VIEW view_subject_averages AS
SELECT
        Sb.subject_id,
        Sb.subject_name,
        ROUND(AVG(M.score), 2) AS average_score,
        COUNT(M.mark_id)    AS marks_recorded
FROM Subjects Sb
LEFT JOIN Marks M ON Sb.subject_id = M.subject_id
GROUP BY Sb.subject_id, Sb.subject_name;
```

```
-- 4.3 Per-student totals (includes students with zero marks)
CREATE OR REPLACE VIEW view_student_totals AS
SELECT
        St.student_id,
        CONCAT(St.first_name, ' ', St.last_name) AS student_name,
        St.enrollment_year,
        COUNT(M.mark_id)            AS marks_count,
        COALESCE(SUM(M.score), 0)           AS total_score,
        ROUND(AVG(M.score), 2)              AS average_score
FROM Students St
LEFT JOIN Marks M ON St.student_id = M.student_id
GROUP BY St.student_id, St.first_name, St.last_name, St.enrollment_year;

-- 4.4 Pass / Fail report (score >= 70 is a Pass)
CREATE OR REPLACE VIEW view_pass_fail AS
SELECT
        CONCAT(St.first_name, ' ', St.last_name) AS student_name,
        Sb.subject_name,
        E.exam_name,
        M.score,
        CASE WHEN M.score >= 70 THEN 'Pass' ELSE 'Fail' END AS status
FROM Marks M
JOIN Students St ON M.student_id = St.student_id
JOIN Subjects Sb ON M.subject_id = Sb.subject_id
JOIN Exams E   ON M.exam_id          = E.exam_id;

-- 4.5 Exam-wise summary (total score, pass count, fail count)
CREATE OR REPLACE VIEW view_exam_summary AS
```

```
SELECT
        E.exam_id,
        E.exam_name,
        SUM(M.score)                              AS total_score,
        SUM(CASE WHEN M.score >= 70 THEN 1 ELSE 0 END)      AS pass_count,
        SUM(CASE WHEN M.score < 70  THEN 1 ELSE 0 END)      AS fail_count
FROM Exams E
JOIN Marks M ON E.exam_id = M.exam_id
GROUP BY E.exam_id, E.exam_name;
```

STORED PROCEDURES

```
-- 5.1 All marks for one student (by student_id)
DROP PROCEDURE IF EXISTS sp_get_student_marks;
DELIMITER $$
CREATE PROCEDURE sp_get_student_marks(IN p_student_id INT)
BEGIN
        SELECT Sb.subject_name, E.exam_name, M.score
        FROM Marks M
        JOIN Subjects Sb ON M.subject_id = Sb.subject_id
        JOIN Exams E   ON M.exam_id          = E.exam_id
        WHERE M.student_id = p_student_id
        ORDER BY E.exam_date, Sb.subject_name;
END $$
DELIMITER ;

-- 5.2 Add a new mark (validates score range 0-100, auto-generates mark_id)
```

```
DROP PROCEDURE IF EXISTS sp_add_mark;
DELIMITER $$
CREATE PROCEDURE sp_add_mark(
        IN p_student_id INT,
        IN p_subject_id INT,
        IN p_exam_id  INT,
        IN p_score      INT
)
BEGIN
        DECLARE v_new_id INT;

        IF p_score < 0 OR p_score > 100 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Score must be between 0 and 100';
        ELSE
        SELECT COALESCE(MAX(mark_id), 0) + 1 INTO v_new_id FROM Marks;
        INSERT INTO Marks (mark_id, student_id, subject_id, exam_id, score)
        VALUES (v_new_id, p_student_id, p_subject_id, p_exam_id, p_score);
        END IF;
END $$
DELIMITER ;

-- 5.3 Average score for one subject (by subject_id)
DROP PROCEDURE IF EXISTS sp_subject_average;
DELIMITER $$
CREATE PROCEDURE sp_subject_average(IN p_subject_id INT)
BEGIN
        SELECT Sb.subject_name, ROUND(AVG(M.score), 2) AS average_score
```

```
        FROM Subjects Sb
        LEFT JOIN Marks M ON Sb.subject_id = M.subject_id
        WHERE Sb.subject_id = p_subject_id
        GROUP BY Sb.subject_name;
END $$
DELIMITER ;

-- 5.4 Full exam report: every mark + pass/fail + exam-level pass count
DROP PROCEDURE IF EXISTS sp_exam_report;
DELIMITER $$
CREATE PROCEDURE sp_exam_report(IN p_exam_id INT)
BEGIN
        SELECT CONCAT(St.first_name, ' ', St.last_name) AS student_name,
        Sb.subject_name,
        M.score,
        CASE WHEN M.score >= 70 THEN 'Pass' ELSE 'Fail' END AS status
        FROM Marks M
        JOIN Students St ON M.student_id = St.student_id
        JOIN Subjects Sb ON M.subject_id = Sb.subject_id
        WHERE M.exam_id = p_exam_id
        ORDER BY M.score DESC;
END $$
DELIMITER ;

-- 5.5 Student report card (all marks + overall average)
DROP PROCEDURE IF EXISTS sp_student_report_card;
DELIMITER $$
CREATE PROCEDURE sp_student_report_card(IN p_student_id INT)
```

```
BEGIN
        SELECT Sb.subject_name, E.exam_name, M.score,
        CASE WHEN M.score >= 70 THEN 'Pass' ELSE 'Fail' END AS status
        FROM Marks M
        JOIN Subjects Sb ON M.subject_id = Sb.subject_id
        JOIN Exams E   ON M.exam_id          = E.exam_id
        WHERE M.student_id = p_student_id;

        SELECT ROUND(AVG(score), 2) AS overall_average
        FROM Marks
        WHERE student_id = p_student_id;
END $$
DELIMITER ;
```

TRIGGERS

```
-- 6.0 Supporting audit table for mark changes
CREATE TABLE marks_log (
        log_id      INT AUTO_INCREMENT PRIMARY KEY,
        mark_id     INT,
        student_id  INT,
        action_type VARCHAR(10),
        old_score   INT,
        new_score   INT,
        change_date DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 6.1 Reject an out-of-range score before it is ever inserted
DROP TRIGGER IF EXISTS trg_validate_score_insert;
DELIMITER $$
CREATE TRIGGER trg_validate_score_insert
BEFORE INSERT ON Marks
FOR EACH ROW
BEGIN
        IF NEW.score < 0 OR NEW.score > 100 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Score must be between 0 and 100';
        END IF;
END $$
DELIMITER ;

-- 6.2 Reject an out-of-range score on UPDATE too
DROP TRIGGER IF EXISTS trg_validate_score_update;
DELIMITER $$
CREATE TRIGGER trg_validate_score_update
BEFORE UPDATE ON Marks
FOR EACH ROW
BEGIN
        IF NEW.score < 0 OR NEW.score > 100 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Score must be between 0 and 100';
        END IF;
END $$
DELIMITER ;
```

```
-- 6.3 Block duplicate marks (same student + subject + exam already exists)
DROP TRIGGER IF EXISTS trg_prevent_duplicate_mark;
DELIMITER $$
CREATE TRIGGER trg_prevent_duplicate_mark
BEFORE INSERT ON Marks
FOR EACH ROW
BEGIN
        DECLARE v_count INT;
        SELECT COUNT(*) INTO v_count
        FROM Marks
        WHERE student_id = NEW.student_id
        AND subject_id = NEW.subject_id
        AND exam_id  = NEW.exam_id;

        IF v_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'This student already has a mark for this subject in this exam';
        END IF;
END $$
DELIMITER ;

-- 6.4 Log every new mark inserted
DROP TRIGGER IF EXISTS trg_log_mark_insert;
DELIMITER $$
CREATE TRIGGER trg_log_mark_insert
AFTER INSERT ON Marks
FOR EACH ROW
BEGIN
```

```
    INSERT INTO marks_log (mark_id, student_id, action_type, old_score, new_score)
    VALUES (NEW.mark_id, NEW.student_id, 'INSERT', NULL, NEW.score);
END $$
DELIMITER ;

-- 6.5 Log every score change (old vs new)
DROP TRIGGER IF EXISTS trg_log_mark_update;
DELIMITER $$
CREATE TRIGGER trg_log_mark_update
AFTER UPDATE ON Marks
FOR EACH ROW
BEGIN
    IF OLD.score <> NEW.score THEN
    INSERT INTO marks_log (mark_id, student_id, action_type, old_score, new_score)
    VALUES (NEW.mark_id, NEW.student_id, 'UPDATE', OLD.score, NEW.score);
    END IF;
END $$
DELIMITER ;

-- 6.6 Log every deleted mark
DROP TRIGGER IF EXISTS trg_log_mark_delete;
DELIMITER $$
CREATE TRIGGER trg_log_mark_delete
AFTER DELETE ON Marks
FOR EACH ROW
BEGIN
    INSERT INTO marks_log (mark_id, student_id, action_type, old_score, new_score)
    VALUES (OLD.mark_id, OLD.student_id, 'DELETE', OLD.score, NULL);
```

```
END $$
DELIMITER ;
```

DEMO CALLS - Views / Stored Procedures / Triggers

```
-- Run these one at a time to see the objects from PART 4-6 in action.

-- Views:
-- SELECT * FROM view_student_marks;
-- SELECT * FROM view_subject_averages;
-- SELECT * FROM view_student_totals ORDER BY total_score DESC;
-- SELECT * FROM view_pass_fail WHERE status = 'Fail';
-- SELECT * FROM view_exam_summary;

-- Stored Procedures:
-- CALL sp_get_student_marks(1);              -- Aarav Sharma's marks
-- CALL sp_add_mark(4, 103, 1, 72);           -- Ananya gets a Hindi Literature mark
-- CALL sp_subject_average(101);              -- Mathematics average
-- CALL sp_exam_report(2);                    -- Final Annual 2024 report
-- CALL sp_student_report_card(2);            -- Diya Patel's report card

-- Triggers (fire automatically, nothing to CALL):
--   * Inserting a Mark with score < 0 or > 100 is blocked      -> trg_validate_score_insert
--   * Updating a Mark to an out-of-range score is blocked    -> trg_validate_score_update
--   * Inserting a duplicate (same student+subject+exam) is
--         blocked                                  -> trg_prevent_duplicate_mark
--   * Every INSERT / UPDATE / DELETE on Marks is written to
--         marks_log automatically                          -> trg_log_mark_insert /
```

-- trg_log_mark_update /

-- trg_log_mark_delete

-- Check the audit trail after running the demo calls above:

-- SELECT * FROM marks_log ORDER BY log_id;

SQL QUERY BANK –

LEVEL 1: SIMPLE QUERIES (Q1–Q3)

Q1. Retrieve All Students: Display the first name and last name of all students.

Q2. Filter by Score: Retrieve all records from the Marks table where the score is greater than 80.

Q3. Sort Data: Display all exams ordered by exam date, starting with the most recent exam.

LEVEL 2: INTERMEDIATE QUERIES (Q4–Q6)

Q4. Student Marks: Display the student's first name, subject name, and score for every recorded mark.

Q5. Subject Averages: Calculate the average score for each subject.

Q6. Count Exams: Find the total number of exams taken by the student Aarav Sharma.

LEVEL 3: COMPLEX QUERIES (Q7–Q9)

Q7. Top Performer: Retrieve the name of the student who obtained the highest individual score in any exam.

Q8. Scores by Exam: Display each exam name and its total score. Show only exams where the total score is greater than 250.

Q9. Pass/Fail Label: Display the student name, subject name, score, and pass/fail status. Consider a score of 70 or above as Pass; otherwise, Fail.

NEW QUERIES (Q10–Q21)

Q10. Students Enrolled in 2023: Retrieve the first and last names of students who enrolled in 2023.

Q11. Highest Score per Subject: Find the highest score recorded in each subject.

Q12. Students Who Took Final Annual Exam: Retrieve the names of students who appeared for the 'Final Annual 2024' examination.

Q13. Students with Average Score Above 80: Display students whose average score across all recorded marks is greater than 80.

Q14. Student Count per Enrollment Year: Find the number of students enrolled in each year.

Q15. Subject with Highest Average: Identify the subject with the highest average score.

Q16. Above-Average Subject Scores: Retrieve students who scored above the average score of their respective subjects.

Q17. Pass and Fail Count per Exam: Calculate the number of students who passed and failed each exam. Consider 70 as the minimum passing score.

Q18. Complete Student Marks Details: Display student name, subject name, exam name, and score for every recorded mark.

Q19. Students Who Never Took Any Exam: Retrieve the names of students who do not have any marks recorded.

Q20. Rank Students by Total Score: Calculate the total score obtained by each student and display students in descending order of their total scores. Include students who have not received any marks.

Q21. Second-Highest Individual Score: Find the second-highest distinct score recorded in the Marks table.